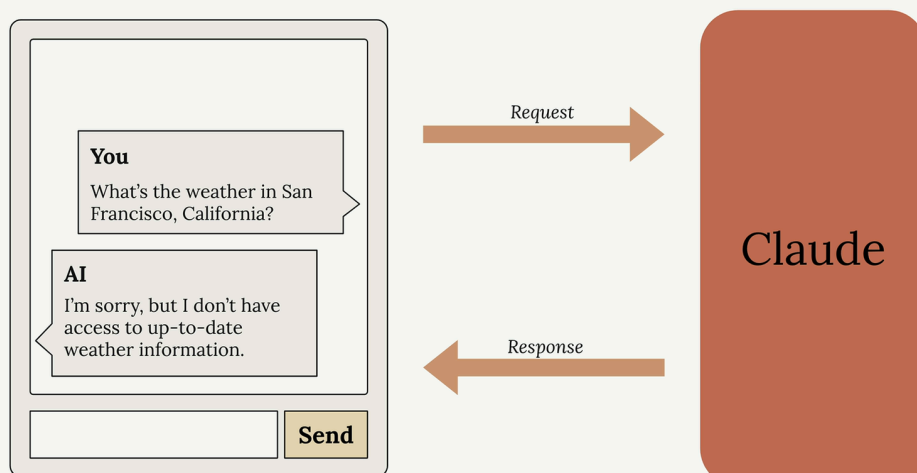


Introducing tool use

Tools allow Claude to access information from the outside world, extending its capabilities beyond what it learned during training. By default, Claude only knows information from its training data and can't access current events, real-time data, or external systems. Tool use solves this limitation by creating a structured way for Claude to request and receive fresh information.

The Problem Without Tools

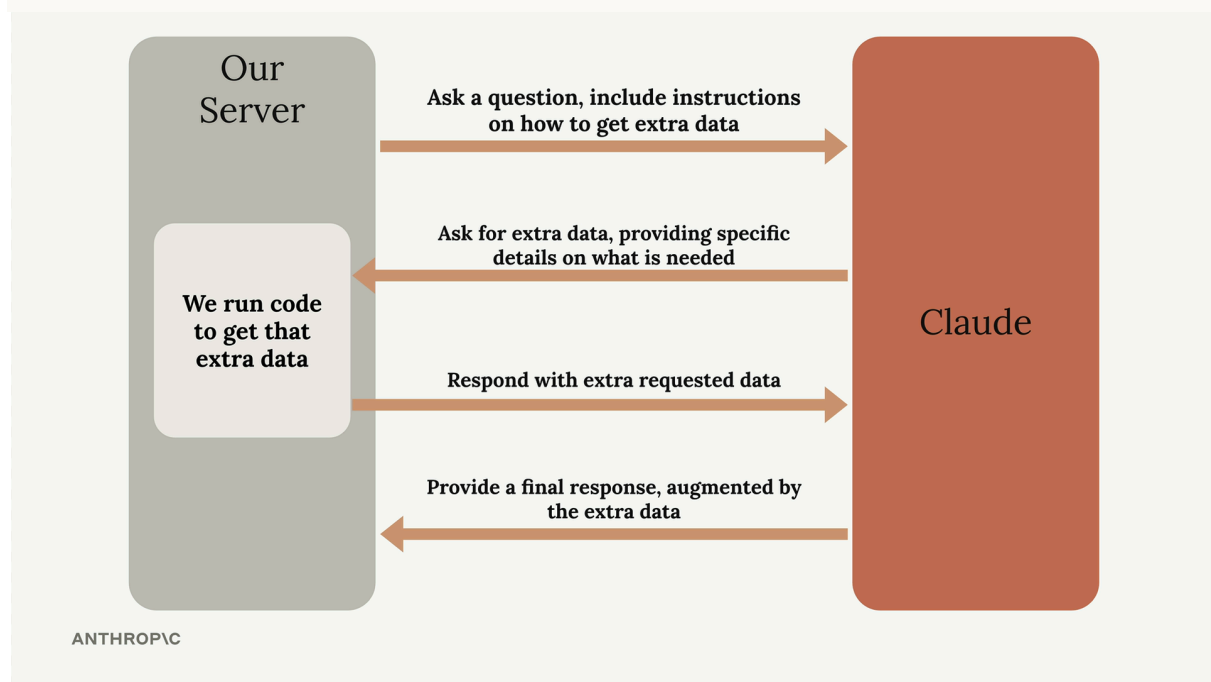
When users ask Claude for current information, it hits a wall. For example, if someone asks "What's the weather in San Francisco, California?" Claude has to respond with something like "I'm sorry, but I don't have access to up-to-date weather information."



This creates a frustrating user experience when people need real-time data that Claude could theoretically help with if it just had access to current information.

How Tool Use Works

Tool use follows a specific back-and-forth pattern between your application and Claude. Here's the complete flow:

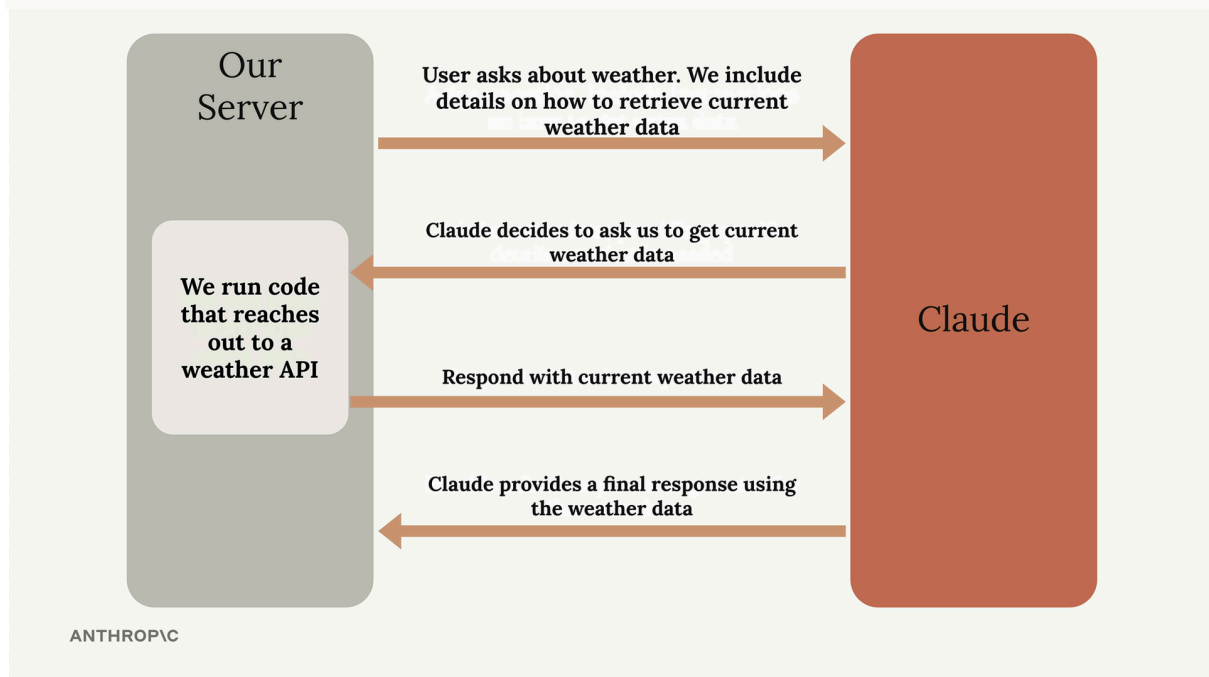


1. Initial Request: You send Claude a question along with instructions on how to get extra data from external sources
2. Tool Request: Claude analyzes the question and decides it needs additional information, then asks for specific details about what data it needs
3. Data Retrieval: Your server runs code to fetch the requested information from external APIs or databases

4. Final Response: You send the retrieved data back to Claude, which then generates a complete response using both the original question and the fresh data

Weather Example in Practice

Let's see how this works with the weather question. The process becomes much more specific:



When a user asks about current weather, you include instructions in your prompt about how to retrieve weather data. Claude recognizes it needs current information and requests weather data for the specific location. Your server then calls a weather API to get real-time conditions and sends that data back to Claude. Finally, Claude combines the fresh weather data with the user's question to provide an accurate, current response.

Key Benefits

- Real-time Information: Access current data that wasn't available during Claude's training
- External System Integration: Connect Claude to databases, APIs, and other services
- Dynamic Responses: Provide answers based on the latest available information
- Structured Interaction: Claude knows exactly what information it needs and how to ask for it

Tool use transforms Claude from a static knowledge base into a dynamic assistant that can work with live data. This opens up possibilities for building applications that need current information, whether that's weather data, stock prices, database queries, or any other real-time information your users might need.

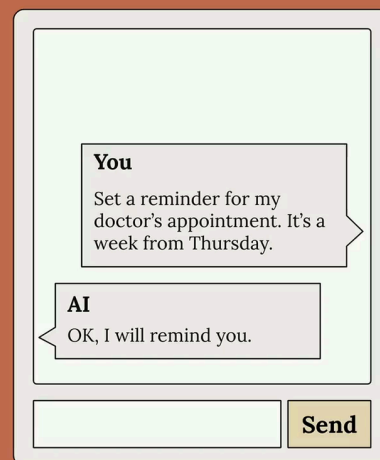
Project overview

We're going to build a practical project that teaches Claude how to set reminders for future dates. This might sound simple at first, but it reveals several interesting challenges that we'll solve using custom tools.

Goal for Tool Use

- We want to teach Claude to set reminders
- Will require us to implement 3-4 custom tools
- For now, we'll focus on just one tool

ANTHROPIC



The goal is straightforward: we want to be able to tell Claude "Set a reminder for my doctor's appointment. It's a week from Thursday" and have Claude respond with "OK, I will remind you." But to make this work, we need to address some limitations in how Claude handles time and reminders.

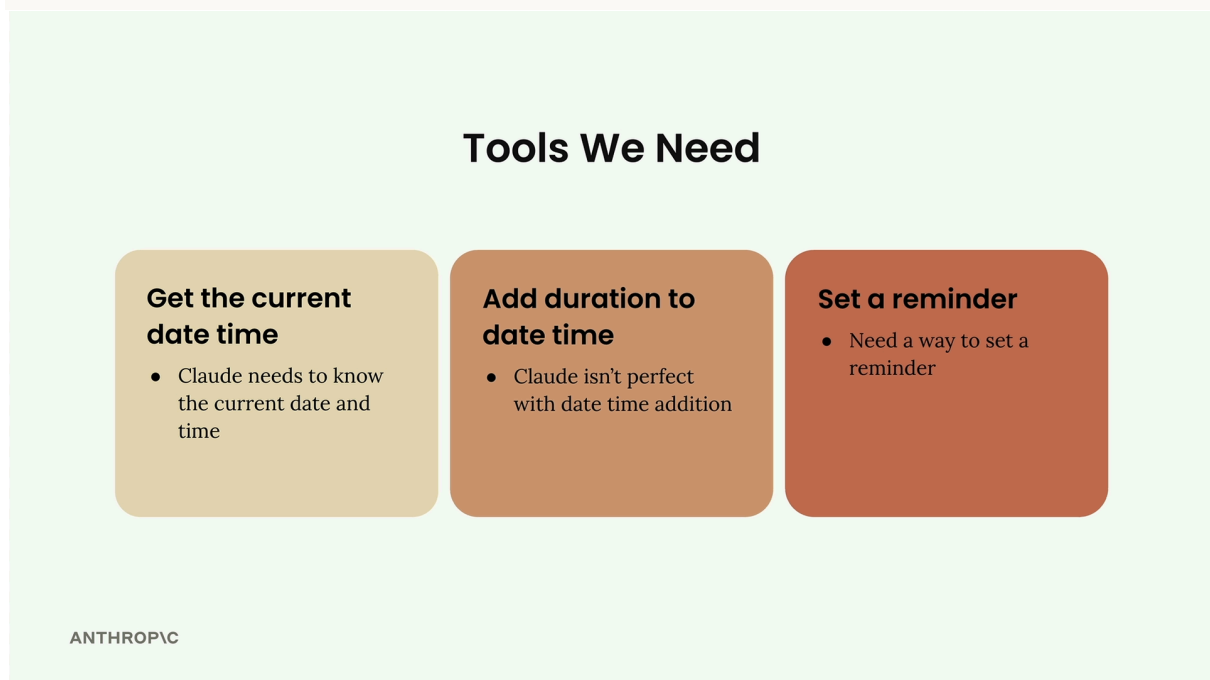
Why This Is Challenging

While Claude knows the current date, there are three specific problems we need to solve:

- Limited time awareness: Claude might know the current date, but not the exact time
- Date calculation issues: Claude doesn't always handle time-based addition well, especially when looking many days into the future
- No reminder capability: Claude doesn't know how to set a reminder
 - it has no built-in mechanism for this

Each of these limitations represents a gap between what Claude can do naturally and what we need for our reminder system. Tools are how we bridge these gaps.

Tools We Need



We'll create three separate tools to handle each challenge:

- Get the current date time: Claude needs to know the current date and time precisely
- Add duration to date time: Claude isn't perfect with date time addition, so we'll give it a reliable tool for this
- Set a reminder: We need a way to actually set a reminder in the system

We'll implement these tools one at a time, starting with the simplest one. This approach lets us understand how tool calling works before building more complex functionality. By the end, Claude will be able to handle natural language requests like "remind me in a week" by combining these tools to calculate the exact time and set the reminder.

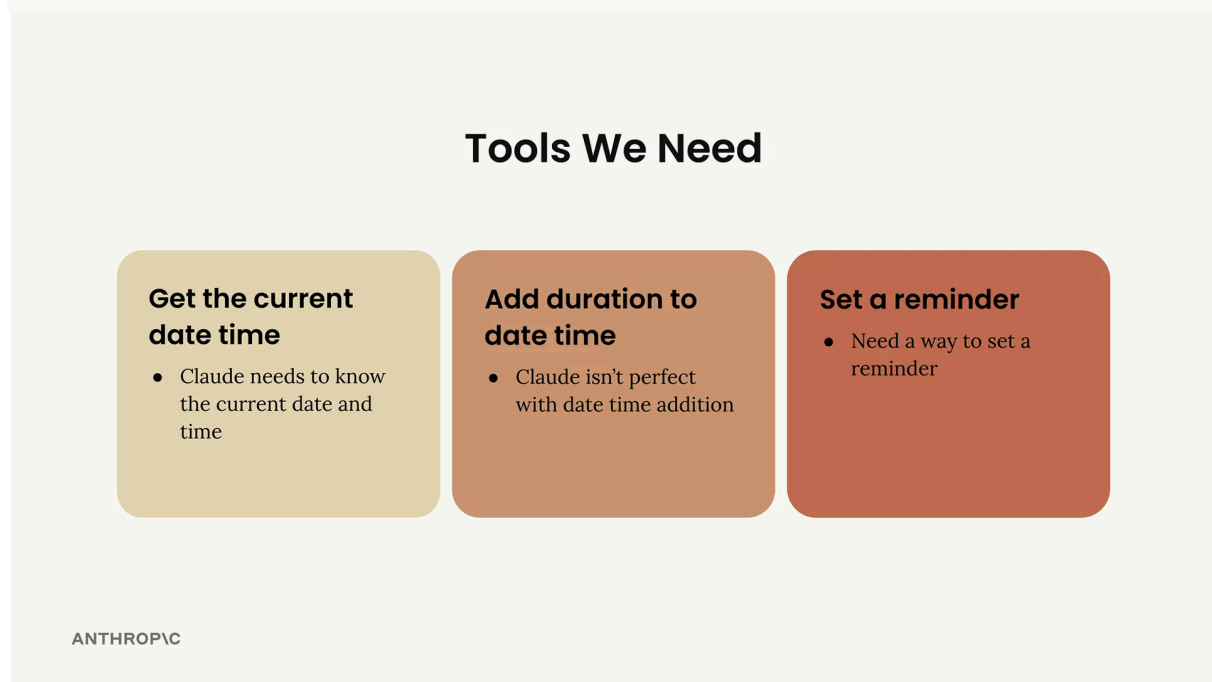
This project demonstrates a key principle of working with AI: when the model has limitations, we extend its capabilities through tools rather than trying to work around those limitations in our prompts.

Tool functions

Downloads

001_tools.ipynb

When building AI applications with Claude, you'll often need to give it access to real-time information or the ability to perform actions. This is where tool functions come in - they're Python functions that Claude can call when it needs additional data to help users.



The image above shows three essential tools we'll be implementing: getting the current date/time, adding duration to dates, and setting reminders. Let's start with the first one.

What Are Tool Functions?

A tool function is a plain Python function that gets executed automatically when Claude decides it needs extra information to help a user. For example, if someone asks "What time is it?", Claude would call your date/time tool to get the current time.

Write a tool function

Write a JSON Schema

Call Claude with JSON Schema

Run Tool

Add Tool Result and call Claude again

Tool Function

- Plain Python function that will be executed when Claude decides it needs some additional information to help the user
- **Best practices**
 - Use well-named, descriptive arguments
 - Validate the inputs, raising an error if they fail validation
 - Return meaningful errors - Claude will try to call to use your function a second time!

```
def get_weather(location):  
    if not location or location.strip() == "":  
        raise ValueError("Location cannot be empty")  
  
    url = "https://fakeweatherapi.example.com/current"  
    params = {  
        "q": location,  
        "appid": api_key,  
        "units": "metric"  
    }  
  
    response = requests.get(url, params=params, timeout=10)  
    response.raise_for_status()  
  
    return response.json()
```

ANTHROPIC

Here's an example of a weather tool function. Notice how it validates inputs and provides clear error messages - these are important best practices.

Best Practices for Tool Functions

When writing tool functions, follow these guidelines:

- Use descriptive names: Both your function name and parameter names should clearly indicate their purpose

- **Validate inputs:** Check that required parameters aren't empty or invalid, and raise errors when they are
- **Provide meaningful error messages:** Claude can see error messages and might retry the function call with corrected parameters

The validation is particularly important because Claude learns from errors. If you raise a clear error like "Location cannot be empty", Claude might try calling the function again with a proper location value.

Building Your First Tool Function

Let's create a function to get the current date and time. This function will accept a date format parameter so Claude can request the time in different formats:

```
def get_current_datetime(date_format="%Y-%m-%d %H:%M:%S") :  
    if not date_format:  
        raise ValueError("date_format cannot be empty")  
  
    return datetime.now().strftime(date_format)
```

This function uses Python's datetime module to get the current time and format it according to the provided format string. The default format gives us year-month-day hour:minute:second.

You can test it with different formats:

```
# Default format: "2024-01-15 14:30:25"  
get_current_datetime()
```

```
# Just hour and minute: "14:30"
```

```
get_current_datetime("%H:%M")
```

The validation check ensures Claude can't pass an empty string for the date format. While this specific error is unlikely, it demonstrates the pattern of validating inputs and providing helpful error messages that Claude can learn from.

Next Steps

Creating the function is just the first step. Next, you'll need to write a JSON schema that describes the function to Claude, then integrate it into your chat system. This tool function approach gives Claude powerful capabilities while keeping your code organized and maintainable.

Tool schemas

After writing your tool function, the next step is creating a JSON schema that tells Claude what arguments your function expects and how to use it. This schema acts as documentation that Claude reads to understand when and how to call your tools.

Understanding JSON Schema

JSON Schema isn't specific to AI or tool calling - it's a widely-used data validation specification that's been around for years. The AI community adopted it because it's a convenient way to describe function parameters and validate data.

Write a tool function

Write a JSON Schema

Call Claude with JSON Schema

Run Tool

Add Tool Result and call Claude again

Write a JSON Schema spec to describe your function

- The JSON Schema helps Claude understand what arguments your function requires
- JSON Schema is not just a LLM thing - this is a commonly used for data validation
 - There are tons of tools online that can help you generate a JSON Schema
- **Best practices**
 - Explain what the tool does, when to use it, and what it returns
 - Aim for 3 to 4 sentences
 - Provide super detailed descriptions.

ANTHROPIC

```
{
  "name": "get_weather",
  "description": "Retrieves current weather...",
  "input_schema": {
    "type": "object",
    "properties": {
      "location": {
        "type": "string",
        "description": "The location for which..."
      }
    },
    "required": [
      "location"
    ]
  }
}
```

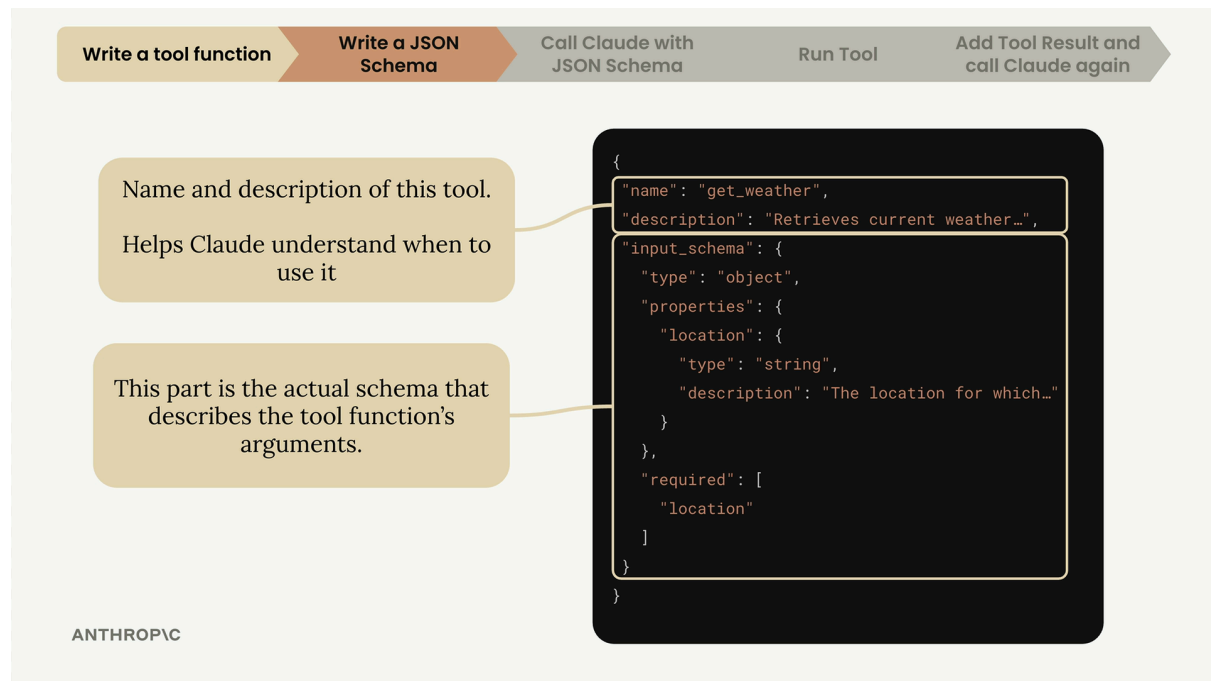
The complete tool specification has three main parts:

- name - A clear, descriptive name for your tool (like "get_weather")
- description - What the tool does, when to use it, and what it returns
- input_schema - The actual JSON schema describing the function's arguments

Writing Effective Descriptions

Your tool description is crucial for helping Claude understand when to use your function. Best practices include:

- Aim for 3-4 sentences explaining what the tool does
- Describe when Claude should use it
- Explain what kind of data it returns
- Provide detailed descriptions for each argument

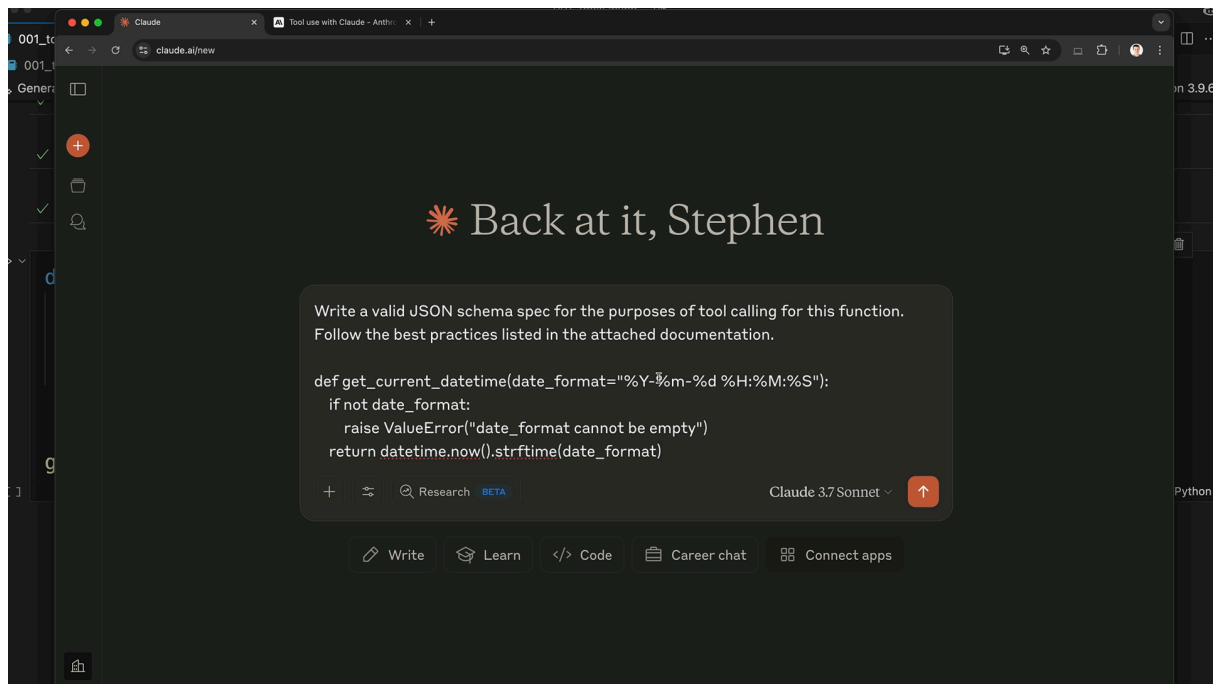


The Easy Way to Generate Schemas

Instead of writing JSON schemas from scratch, you can use Claude itself to generate them. Here's the process:

1. Copy your tool function code
2. Go to Claude and ask it to write a JSON schema for tool calling
3. Include the Anthropic documentation on tool use as context
4. Let Claude generate a properly formatted schema following best practices

The prompt should be something like: "Write a valid JSON schema spec for the purposes of tool calling for this function. Follow the best practices listed in the attached documentation."



Implementing the Schema in Code

Once Claude generates your schema, copy it into your code file. Here's a good naming pattern to follow:

```
def get_current_datetime(date_format="%Y-%m-%d %H:%M:%S") :
    if not date_format:
        raise ValueError("date_format cannot be empty")
    return datetime.now().strftime(date_format)
```

```
get_current_datetime_schema = {
    "name": "get_current_datetime",
    "description": "Returns the current date and time
formatted according to the specified format",
    "input_schema": {
        "type": "object",
        "properties": {
            "date_format": {
                "type": "string",
```

```

        "description": "A string specifying the format
of the returned datetime. Uses Python's strftime format
codes.",
        "default": "%Y-%m-%d %H:%M:%S"
    },
    "required": []
}

```

Use the pattern of **function_name** followed by **function_name_schema** to keep your schemas organized and easy to match with their corresponding functions.

Adding Type Safety

For better type checking, import and use the **ToolParam** type from the Anthropic library:

```

from anthropic.types import ToolParam

get_current_datetime_schema = ToolParam({
    "name": "get_current_datetime",
    "description": "Returns the current date and time
formatted according to the specified format",
    # ... rest of schema
})

```

While not strictly necessary for functionality, this prevents type errors when you use the schema with Claude's API and makes your code more robust.

Handling message blocks

When working with Claude's tool functionality, you'll encounter a new type of response structure that's different from the simple text responses you've seen before. Instead of just getting back a single text block, Claude can now return multi-block messages that contain both text and tool usage information.

Making Tool-Enabled API Calls

To enable Claude to use tools, you need to include a **tools** parameter in your API call. Here's how to structure the request:

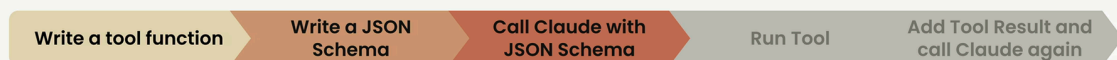
```
messages = []
messages.append({
    "role": "user",
    "content": "What is the exact time, formatted as
HH:MM:SS?"
})

response = client.messages.create(
    model=model,
    max_tokens=1000,
    messages=messages,
    tools=[get_current_datetime_schema],
)
```

The **tools** parameter takes a list of JSON schemas that describe the available functions Claude can call.

Understanding Multi-Block Messages

When Claude decides to use a tool, it returns an assistant message with multiple blocks in the content list. This is a significant change from the simple text-only responses you've worked with before.



Multi-Block messages

Blocks are also often referred to as “parts”

Assistant Message's Content List	"Text" Block	<i>"I can help you find out the current time. Let me find that information for you"</i>
	"ToolUse" Block	<pre>ToolUseBlock(id='toolu_01QtKMDNkU1FBj73NP2FbF8w', input={'date_format': '%H:%M:%S'}, name='get_current_datetime', type='tool_use')</pre>

ANTHROPIC

A multi-block message typically contains:

- Text Block - Human-readable text explaining what Claude is doing (like "I can help you find out the current time. Let me find that information for you")
- ToolUse Block - Instructions for your code about which tool to call and what parameters to use

The ToolUse block includes:

- An ID for tracking the tool call
- The name of the function to call (like "get_current_datetime")

- Input parameters formatted as a dictionary
- The type designation "tool_use"

Managing Conversation History with Multi-Block Messages

Remember that Claude doesn't store conversation history - you need to manage it manually. When working with tool responses, you must preserve the entire content structure, including all blocks.

Here's how to properly append a multi-block assistant message to your conversation history:

```
messages.append({  
    "role": "assistant",  
    "content": response.content  
})
```

This preserves both the text block and the tool use block, which is crucial for maintaining the conversation context when you make subsequent API calls.

The Complete Tool Usage Flow

```
001_tools.ipynb
001_tools.ipynb > # Helper functions
Generate + Code + Markdown | ▶ Run All ⏸ Restart ⚙ Clear All Outputs | Jupyter Variables | Outline ...
✓ # Load env variables and create client ...

# Helper functions
def add_user_message(messages, text):
    user_message = {"role": "user", "content": text}
    messages.append(user_message)

def add_assistant_message(messages, text):
    assistant_message = {"role": "assistant", "content": text}
    messages.append(assistant_message)

def chat(messages, system=None, temperature=1.0, stop_sequences=[]):
    params = {
        "model": model,
        "max_tokens": 1000,
        "messages": messages,
        "temperature": temperature,
```

The tool usage process follows this pattern:

1. Send user message with tool schema to Claude
2. Receive assistant message with text block and tool use block
3. Extract tool information and execute the actual function
4. Send tool result back to Claude along with complete conversation history
5. Receive final response from Claude

Each step requires careful handling of the message structure to ensure Claude has the full context it needs to provide accurate responses.

Updating Helper Functions

If you've been using helper functions like `add_user_message()` and `add_assistant_message()`, you'll need to update them to handle

multi-block content. The current versions likely only support single text blocks, but now they need to accommodate the more complex content structures that include tool use blocks.

This multi-block message handling is essential for building robust applications that can seamlessly integrate Claude's tool capabilities while maintaining proper conversation flow.

Sending tool results

After Claude requests a tool call, you need to execute the function and send the results back. This completes the tool use workflow by providing Claude with the information it requested.

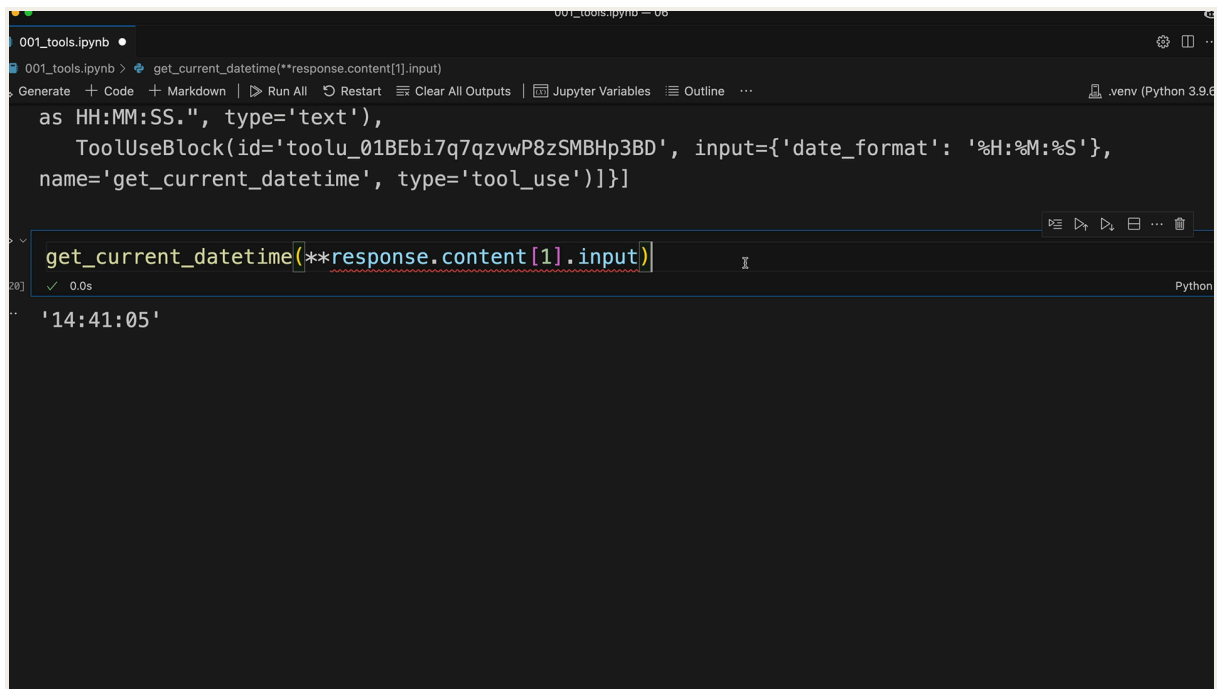
Running the Tool Function

When Claude responds with a tool use block, you extract the input parameters and call your function. Here's how to access the tool parameters:

```
response.content[1].input
```

This gives you a dictionary of the arguments Claude wants to pass to your function. Since your function expects keyword arguments rather than a dictionary, you use Python's unpacking syntax:

```
get_current_datetime(**response.content[1].input)
```



The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar indicates the file is '001_tools.ipynb' and the environment is '.venv (Python 3.9.6)'. The code cell contains the following Python code:

```
as HH:MM:SS.", type='text'),  
    ToolUseBlock(id='toolu_01BEbi7q7qzvwP8zSMBHp3BD', input={'date_format': '%H:%M:%S'},  
name='get_current_datetime', type='tool_use'}}])
```

The code cell is followed by an output cell showing the result of the function call:

```
get_current_datetime(**response.content[1].input)
```

The output is a string representing the current time in HH:MM:SS format:

```
'14:41:05'
```

Tool Result Block

After running the tool function, you need to send the results back to Claude using a tool result block. This block goes inside a user message and tells Claude what happened when you executed the tool.

Write a tool function

Write a JSON
Schema

Call Claude with
JSON Schema

Run Tool

Add Tool Result and
call Claude again

Tool Result Block

- Placed in the 'content' list of a user message
- Communicates the results of running a tool function back to Claude
- **"tool_use_id"** - Must match the id of the ToolUse block that this ToolResult corresponds to
- **"content"** - Output from running your tool, serialized as a string
- **"is_error"** - True if an error occurred

```
{  
  "tool_use_id": "toolu_01BEbi7q7qz",  
  "type": "tool_result",  
  "content": "12:47:13",  
  "is_error": False  
}
```

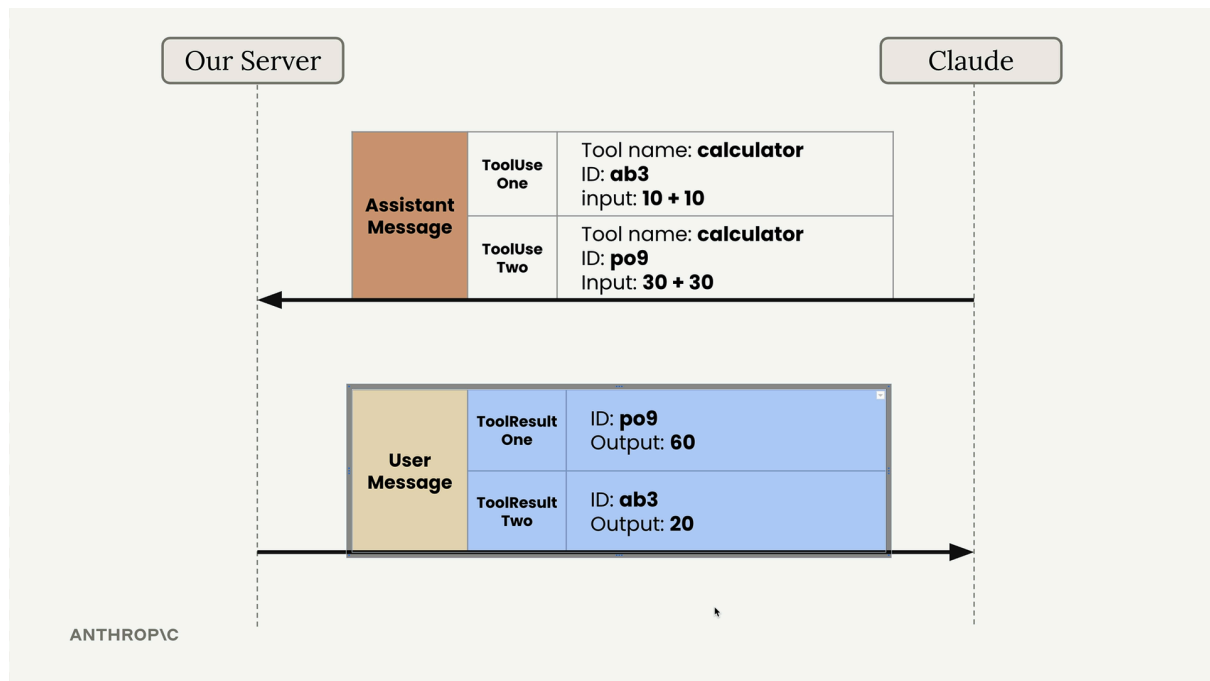
ANTHROPIC

The tool result block has several important properties:

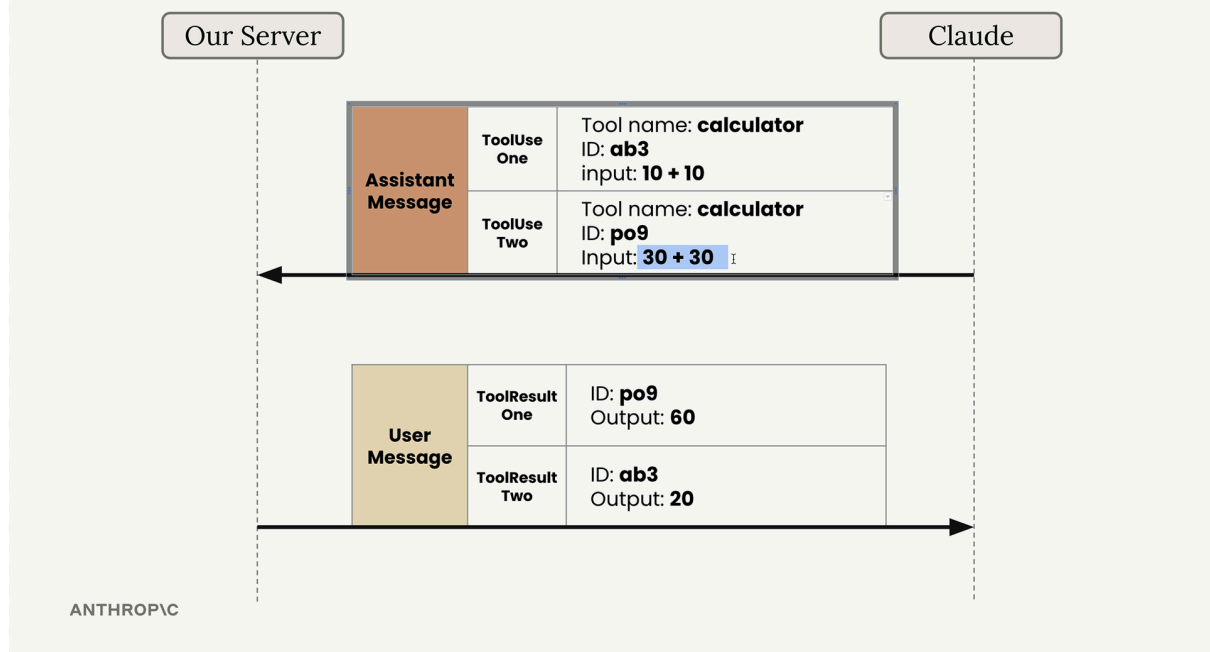
- **tool_use_id** - Must match the id of the ToolUse block that this ToolResult corresponds to
- **content** - Output from running your tool, serialized as a string
- **is_error** - True if an error occurred

Handling Multiple Tool Calls

Claude can request multiple tool calls in a single response. For example, if a user asks "What's 10 + 10 and what's 30 + 30?", Claude might respond with two separate ToolUse blocks.



Each tool call gets a unique ID, and you must match these IDs when sending back results. This ensures Claude knows which result corresponds to which request, even if the results arrive in a different order.



Building the Follow-up Request

Your follow-up request to Claude must include the complete conversation history plus the new tool result. Here's the structure:

```
messages.append({
    "role": "user",
    "content": [{
        "type": "tool_result",
        "tool_use_id": response.content[1].id,
        "content": "15:04:22",
        "is_error": False
    }]
})
```

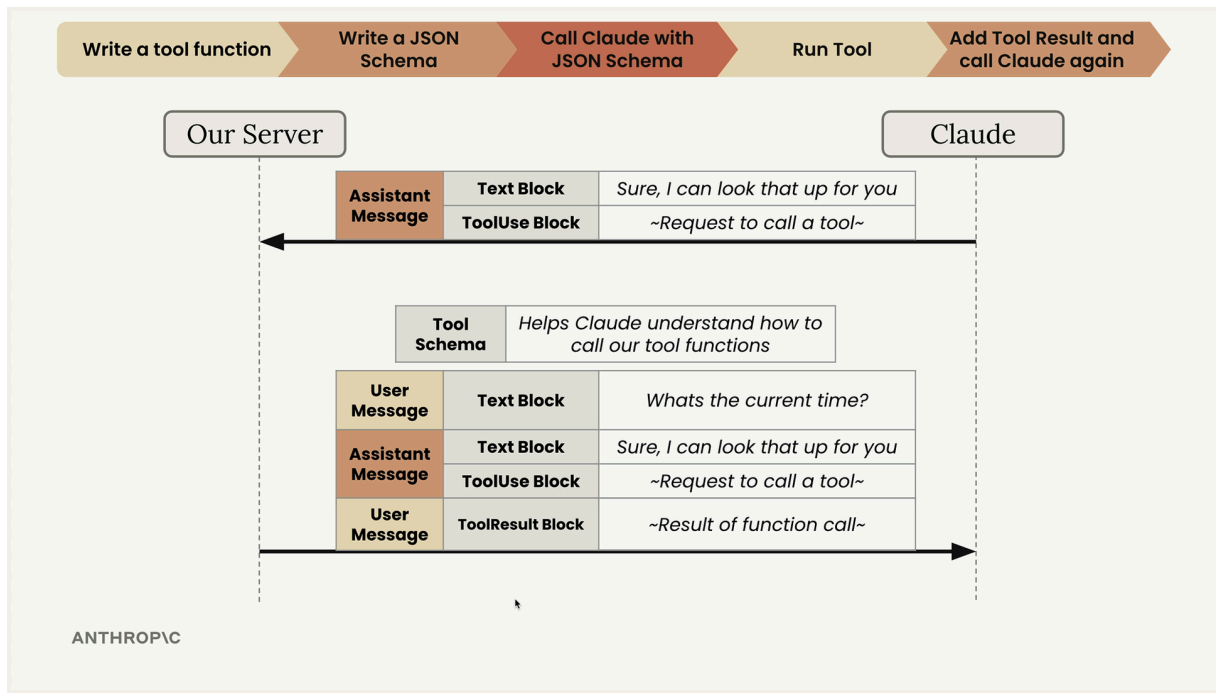
The complete message history now contains:

- Original user message
- Assistant message with tool use block
- User message with tool result block

Making the Final Request

When sending the follow-up request, you must still include the tool schema even though you're not expecting Claude to make another tool call. Claude needs the schema to understand the tool references in your conversation history.

```
client.messages.create(
    model=model,
    max_tokens=1000,
    messages=messages,
    tools=[get_current_datetime_schema]
)
```



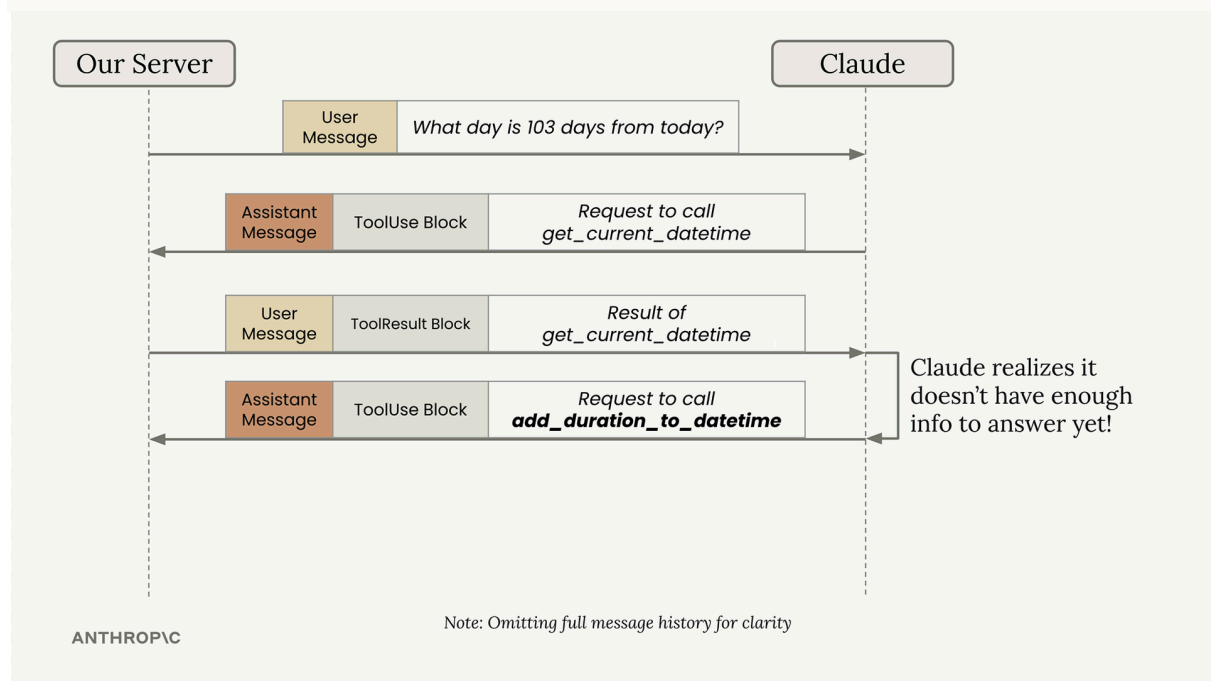
Claude will then respond with a final message that incorporates the tool results into a natural response for the user. The tool use workflow is now complete - you've successfully enabled Claude to access real-time information through your custom function.

Multi-turn conversations with tools

Downloads

001_tools_007.ipynb

When building applications with multiple tools, you need to handle scenarios where Claude might need to call several tools in sequence to answer a single user question. For example, if a user asks "What day is 103 days from today?", Claude needs to first get the current date, then add 103 days to it.

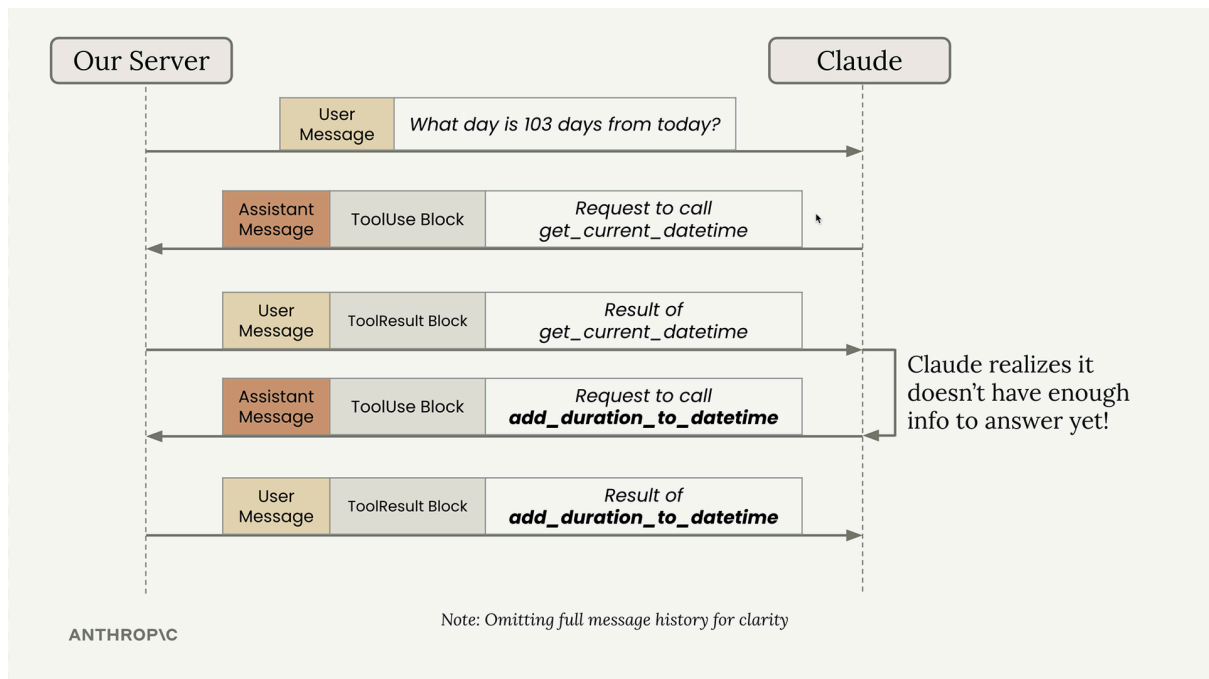


This creates a multi-turn conversation pattern where Claude makes multiple tool requests before providing a final answer. Your application needs to handle this automatically.

The Multi-Turn Tool Pattern

Here's what happens behind the scenes when Claude needs multiple tools:

1. User asks: "What day is 103 days from today?"
2. Claude responds with a tool use block requesting **get_current_datetime**
3. Your server calls the function and returns the result
4. Claude realizes it needs more information and requests **add_duration_to_datetime**
5. Your server calls that function and returns the result
6. Claude now has enough information to provide the final answer



Building a Conversation Loop

To handle this pattern, you need a conversation loop that continues until Claude stops requesting tools:

```
def run_conversation(messages):
    while True:
        response = chat(messages)

        add_assistant_message(messages, response)

        # Pseudo code
        if response isn't asking for a tool:
            break

        tool_result_blocks = run_tools(response)
        add_user_message(messages, tool_result_blocks)

    return messages
```

Conversation with tools

- Provide an initial list of messages
- Feed messages into Claude
- **If Claude isn't asking for a tool use**, then we must have a final answer to send back to our user
- **If Claude wants to use a tool**, then we will run the tool, put the results into a user message, and run Claude again

```
def run_conversation(messages):
    while True:
        response = chat(messages)

        add_user_message(messages, response)

        # Pseudo code
        if response isn't asking for a tool:
            break

        tool_result_blocks = run_tools(response)
        add_user_message(tool_result_blocks)

    return messages
```

ANTHROPIC

Refactoring Helper Functions

Before implementing the conversation loop, you need to update your helper functions to handle multiple message blocks properly.

Updating Message Handlers

Your `add_user_message` and `add_assistant_message` functions currently assume you're always working with plain text. Update them to handle full message objects:

```
from anthropic.types import Message

def add_user_message(messages, message):
    user_message = {
        "role": "user",
        "content": message.content if isinstance(message,
Message) else message
    }

    messages.append(user_message)
```

This allows you to pass in either a string, a list of blocks, or a complete message object.

Updating the Chat Function

Modify your chat function to accept a list of tools and return the full message instead of just text:

```
def chat(messages, system=None, temperature=1.0,
stop_sequences=[], tools=None):
    params = {
        "model": model,
        "max_tokens": 1000,
        "messages": messages,
        "temperature": temperature,
        "stop_sequences": stop_sequences,
    }

    if tools:
        params["tools"] = tools
```

```
if system:
    params["system"] = system

message = client.messages.create(**params)

return message
```

Extracting Text from Messages

Since you're now returning full message objects, create a helper to extract text when needed:

```
def text_from_message(message):
    return "\n".join(
        [block.text for block in message.content if block.type
         == "text"]
    )
```

This function finds all text blocks in a message and joins them together, which is useful when you need to display the final response to users.

Key Improvements

These refactoring steps prepare your code for robust tool handling:

- Flexible message handling - Your helper functions can now work with different message formats
- Tool support in chat - The chat function can receive and pass through tool schemas
- Full message returns - You get complete message objects instead of just text, preserving all blocks

- Text extraction utility - Easy way to get readable text from complex messages

With these foundations in place, you're ready to implement the conversation loop that handles multiple tool calls automatically, creating a seamless experience where Claude can use as many tools as needed to answer user questions.

Implementing multiple turns

Downloads

001_tools_008.ipynb

Building a conversation system with tools requires implementing a loop that keeps calling Claude until it stops requesting tool usage. When Claude no longer asks for tools, that signals it has a final response ready for the user.

Detecting Tool Requests

The key to knowing whether Claude wants to use a tool lies in the **stop_reason** field of the response message. When Claude decides it needs to call a tool, this field gets set to **"tool_use"**. This gives us a clean way to check if we need to continue the conversation loop:

```
if response.stop_reason != "tool_use":  
  
    break # Claude is done, no more tools needed
```

The Conversation Loop

The main conversation function follows a simple pattern:

```
def run_conversation(messages):
```

```
    while True:
        response = chat(messages,
tools=[get_current_datetime_schema])
        add_assistant_message(messages, response)
        print(text_from_message(response))

        if response.stop_reason != "tool_use":
            break

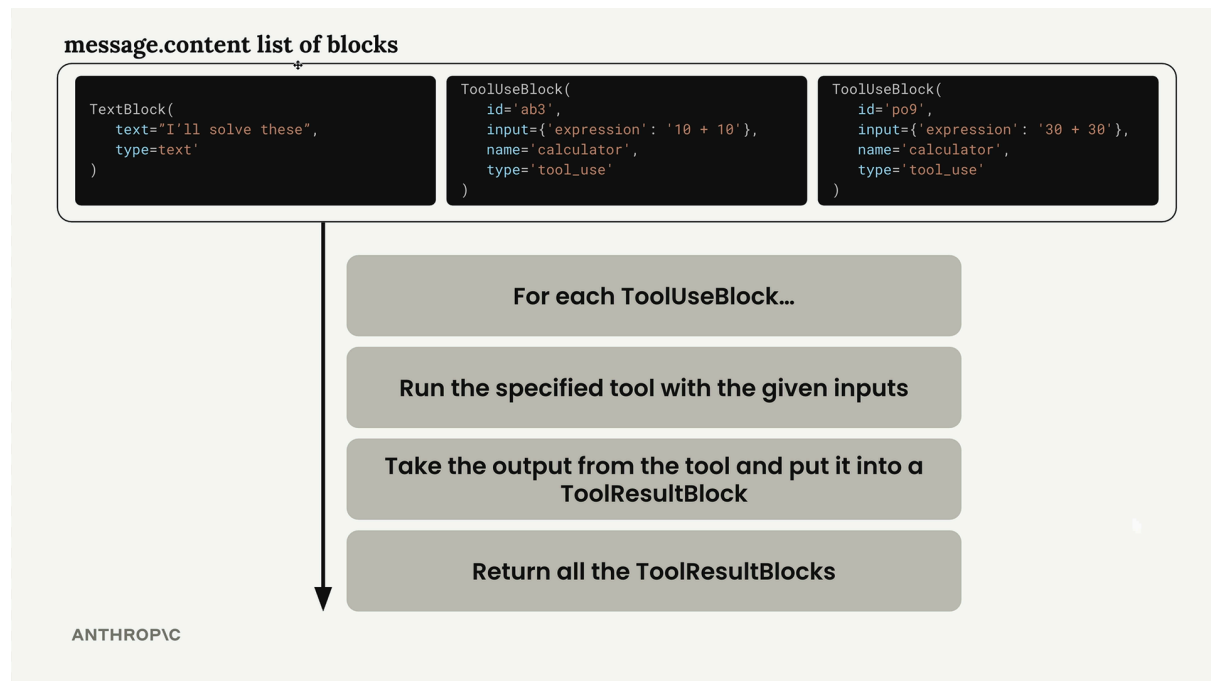
        tool_results = run_tools(response)
        add_user_message(messages, tool_results)

    return messages
```

This loop continues until Claude provides a final answer without requesting any tools.

Handling Multiple Tool Calls

Claude can request multiple tools in a single response. The message content contains a list of blocks, and we need to process each tool use block separately:



The **run_tools** function handles this by filtering for tool use blocks and processing each one:

```
def run_tools(message):
    tool_requests = [
        block for block in message.content if block.type ==
"tool_use"
    ]
    tool_result_blocks = []

    for tool_request in tool_requests:
        # Process each tool request...
```

Tool Result Blocks

Each tool use block must be answered with a corresponding tool result block. The connection between them is maintained through matching IDs:

```
001_tools.ipynb •
001_tools.ipynb > def run_tools(message):
Generate + Code + Markdown | ▶ Run All | ⌂ Restart | 🗑 Clear All Outputs | 📄 Jupyter Variables | 📖 Outline | ⋮
Python 3.9.6

def run_tools(message):
    tool_requests = [
        block for block in message.content if block.type == "tool_use"
    ]
    tool_result_blocks = []

    for tool_request in tool_requests:
        if tool_request.name == "get_current_datetime":
            tool_output = get_current_datetime(**tool_request.input)
            tool_result_block = {
                "type": "tool_result",
                "tool_use_id": tool_request.id,
                "content": json.dumps(tool_output)
            }

def run_conversation(messages):
    while True:
        response = chat(messages, tools=[get_current_datetime_schema])
```

The tool result block structure includes:

```
tool_result_block = {
    "type": "tool_result",
    "tool_use_id": tool_request.id,
    "content": json.dumps(tool_output),
    "is_error": False
}
```

Error Handling

Robust tool execution requires handling potential errors. When a tool fails, we still need to provide a result block to Claude:

```
try:
    tool_output = run_tool(tool_request.name,
    tool_request.input)
    tool_result_block = {
        "type": "tool_result",
        "tool_use_id": tool_request.id,
```

```

        "content": json.dumps(tool_output),
        "is_error": False
    }
except Exception as e:
    tool_result_block = {
        "type": "tool_result",
        "tool_use_id": tool_request.id,
        "content": f"Error: {e}",
        "is_error": True
    }

```

Scalable Tool Routing

To support multiple tools, create a routing function that maps tool names to their implementations:

```

def run_tool(tool_name, tool_input):
    if tool_name == "get_current_datetime":
        return get_current_datetime(**tool_input)
    elif tool_name == "another_tool":
        return another_tool(**tool_input)

    # Add more tools as needed

```

This approach makes it easy to add new tools without modifying the core conversation logic.

Complete Workflow

The complete multi-turn conversation works like this:

- Send user message to Claude with available tools
- Claude responds with text and/or tool requests
- Execute all requested tools and create result blocks
- Send tool results back as a user message
- Repeat until Claude provides a final answer

This creates a seamless experience where Claude can use multiple tools across several turns to fully answer complex user requests. The conversation history maintains the complete context, allowing Claude to build upon previous tool results to provide comprehensive responses.

Using multiple tools

Downloads

001_tools_009.ipynb

Adding multiple tools to your Claude implementation becomes straightforward once you have the core tool-handling infrastructure in place. This tutorial shows how to integrate additional tools by following a simple pattern.

Tools We Need

Get the current date time

- Claude needs to know the current date and time

Add duration to date time

- Claude isn't perfect with date time addition

Set a reminder

- Need a way to set a reminder

ANTHROPIC

The Tools We're Adding

We need three main capabilities for our reminder system:

- Get current date time - Claude needs to know the current date and time
- Add duration to date time - Claude isn't perfect with date time addition
- Set a reminder - Need a way to set a reminder

The good news is that most of the implementation work is already done. The **add_duration_to_datetime** function and **set_reminder** function are provided, along with their corresponding schemas.

Adding Tools to the Conversation

First, update the **run_conversation** function to include the new tool schemas in the tools list:

```
response = chat(messages, tools=[
    get_current_datetime_schema,
    add_duration_to_datetime_schema,
    set_reminder_schema
])
```

This tells Claude about all three available tools it can use during the conversation.

Updating the Tool Router

Next, modify the **run_tool** function to handle the new tool calls. Add `elif` cases for each new tool:

```
def run_tool(tool_name, tool_input):  
    if tool_name == "get_current_datetime":  
        return get_current_datetime(**tool_input)  
    elif tool_name == "add_duration_to_datetime":  
        return add_duration_to_datetime(**tool_input)  
    elif tool_name == "set_reminder":  
  
        return set_reminder(**tool_input)
```

The pattern is simple: check the tool name, call the corresponding function with the provided input, and return the result.

Testing Multiple Tool Usage

To test the system, try a request that requires multiple tools: "Set a reminder for my doctors appointment. Its 177 days after Jan 1st, 2050."

This request forces Claude to:

1. Calculate the date (using **add_duration_to_datetime**)
2. Set the reminder (using **set_reminder**)

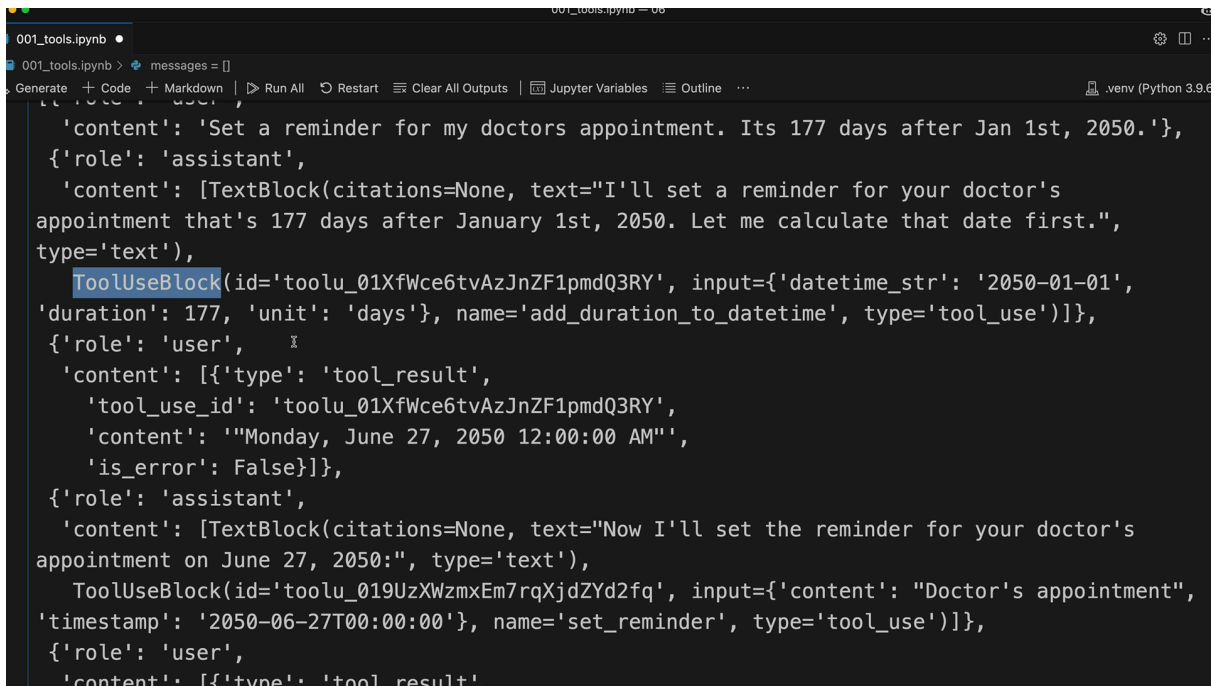
```
001_tools.ipynb •
001_tools.ipynb > messages = []
Generate + Code + Markdown | ▶ Run All ⌂ Restart ⌂ Clear All Outputs | Jupyter Variables ⌂ Outline ...
Python 3.9.6
)
run_conversation(messages)
✓ 6.4s
Python
I'll set a reminder for your doctor's appointment that's 177 days after January 1st, 2050. Let me calculate that date first.
Now I'll set the reminder for your doctor's appointment on June 27, 2050:
-----
Setting the following reminder for 2050-06-27T00:00:00:
Doctor's appointment
-----
Your reminder has been set successfully! You will be reminded of your doctor's appointment on Monday, June 27, 2050. If you need to set a specific time for this appointment rather than midnight (12:00 AM), please let me know and I can adjust the reminder.
[{'role': 'user',
  'content': 'Set a reminder for my doctors appointment. Its 177 days after Jan 1st, 2050.'},
 {'role': 'assistant',
  'content': [TextBlock(citations=None, text="I'll set a reminder for your doctor's appointment that's 177 days after January 1st, 2050. Let me calculate that date first.",
```

Claude handles this by first explaining what it needs to do, then making the appropriate tool calls in sequence. The conversation shows Claude calculating June 27, 2050 as the target date, then setting the reminder for that date.

Understanding the Message Flow

When you examine the conversation history, you'll see the complete message structure:

- User message with the request
- Assistant message containing both text and tool use blocks
- Tool result messages
- Follow-up assistant messages



```

001_tools.ipynb
001_tools.ipynb > messages = []
Generate + Code + Markdown + Run All + Restart + Clear All Outputs + Jupyter Variables + Outline ...
.venv (Python 3.9.6)

{
  'content': 'Set a reminder for my doctors appointment. Its 177 days after Jan 1st, 2050.',
  'role': 'assistant',
  'content': [TextBlock(citations=None, text="I'll set a reminder for your doctor's
appointment that's 177 days after January 1st, 2050. Let me calculate that date first.",
type='text'),
  ToolUseBlock(id='toolu_01XfWce6tvAzJnZF1pmdQ3RY', input={'datetime_str': '2050-01-01',
'duration': 177, 'unit': 'days'}, name='add_duration_to_datetime', type='tool_use')]],
  'role': 'user',
  'content': [{ 'type': 'tool_result',
    'tool_use_id': 'toolu_01XfWce6tvAzJnZF1pmdQ3RY',
    'content': '"Monday, June 27, 2050 12:00:00 AM"',
    'is_error': False}],
  'role': 'assistant',
  'content': [TextBlock(citations=None, text="Now I'll set the reminder for your doctor's
appointment on June 27, 2050:", type='text'),
  ToolUseBlock(id='toolu_019UzXWzmxEm7rqXjdZYd2fq', input={'content': "Doctor's appointment",
'timestamp': '2050-06-27T00:00:00'}, name='set_reminder', type='tool_use')]],
  'role': 'user',
  'content': [{ 'type': 'tool_result'

```

This demonstrates how Claude can include multiple blocks in a single message - combining explanatory text with tool usage requests.

The Simple Pattern for Adding Tools

Once you have the core tool infrastructure, adding new tools follows this pattern:

1. Create the tool function implementation
2. Define the tool schema
3. Add the schema to the tools list in **run_conversation**
4. Add a case for the tool in **run_tool**

This modular approach makes it easy to expand your AI assistant's capabilities without restructuring existing code. Each new tool integrates seamlessly with the existing conversation flow and tool-handling logic.

Fine grained tool calling

Downloads

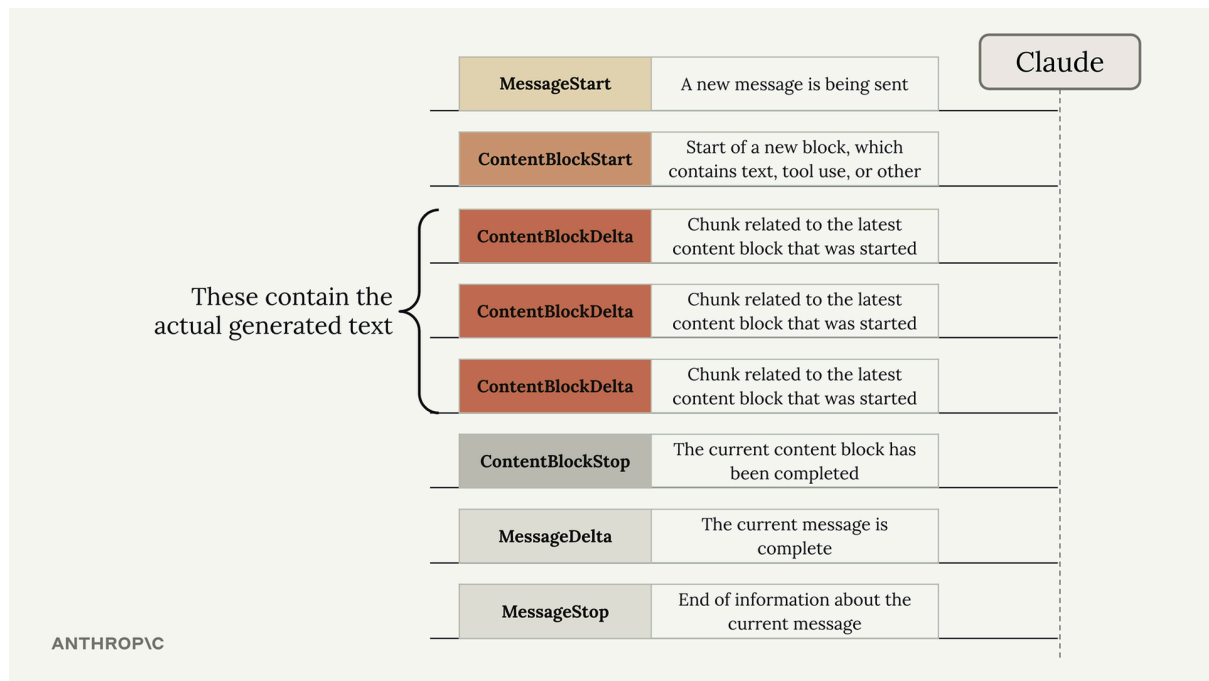
`003_tool_streaming.ipynb` (opens in new tab)

`003_tool_streaming_completed.ipynb`

When you combine tool use with streaming in Claude, you get real-time updates as the AI generates tool arguments. This creates a more responsive user experience, but there are some important details to understand about how it works behind the scenes.

Basic Tool Streaming

With streaming enabled, Claude sends back different types of events as it processes your request. You're already familiar with events like **ContentBlockDelta** for regular text generation. For tool use, you'll also need to handle a new event type called **InputJsonEvent**.

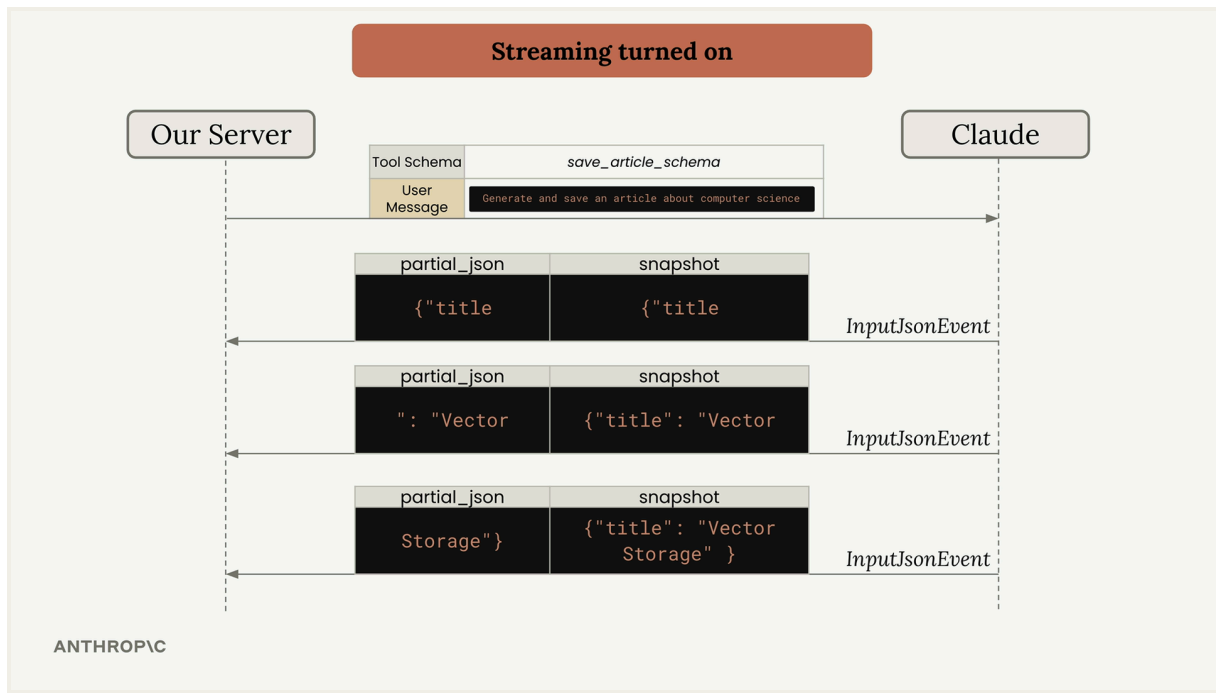


Each **InputJsonEvent** contains two key properties:

- `partial_json` - A chunk of JSON representing part of the tool arguments
- `snapshot` - The cumulative JSON built up from all chunks received so far

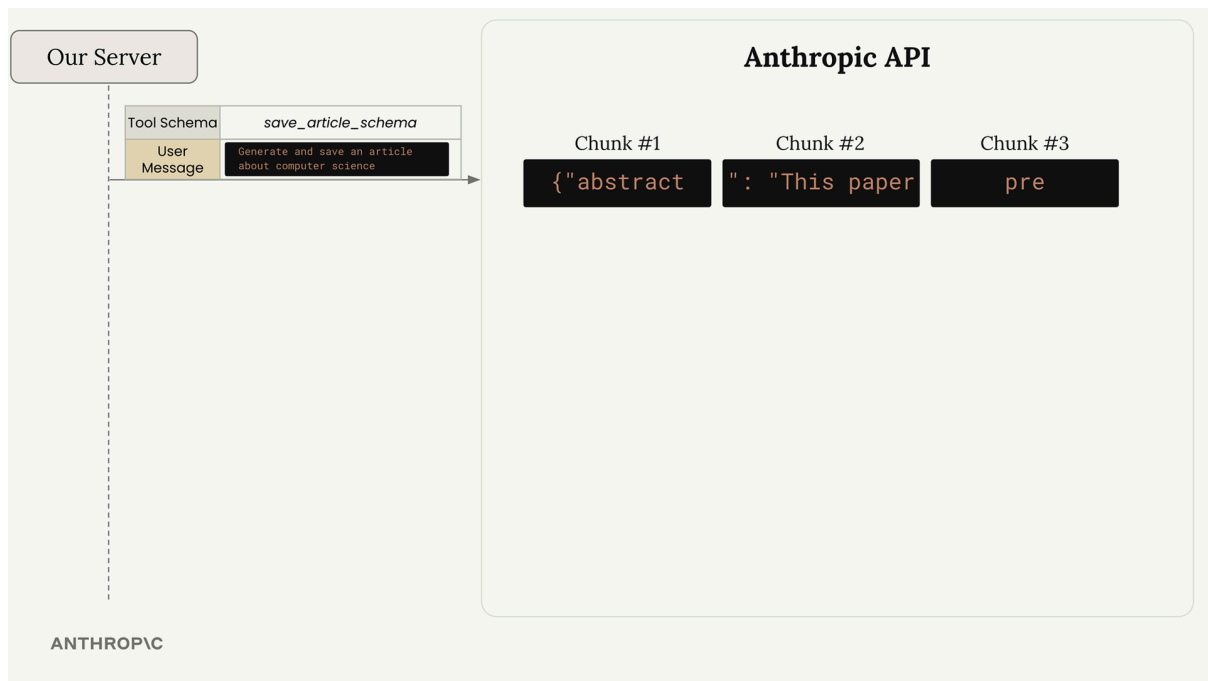
Here's how you handle these events in your streaming pipeline:

```
for chunk in stream:
    if chunk.type == "input_json":
        # Process the partial JSON chunk
        print(chunk.partial_json)
        # Or use the complete snapshot so far
        current_args = chunk.snapshot
```



How JSON Validation Works

Here's where things get interesting. The Anthropic API doesn't immediately send you every chunk as Claude generates it. Instead, it buffers chunks and validates them first.

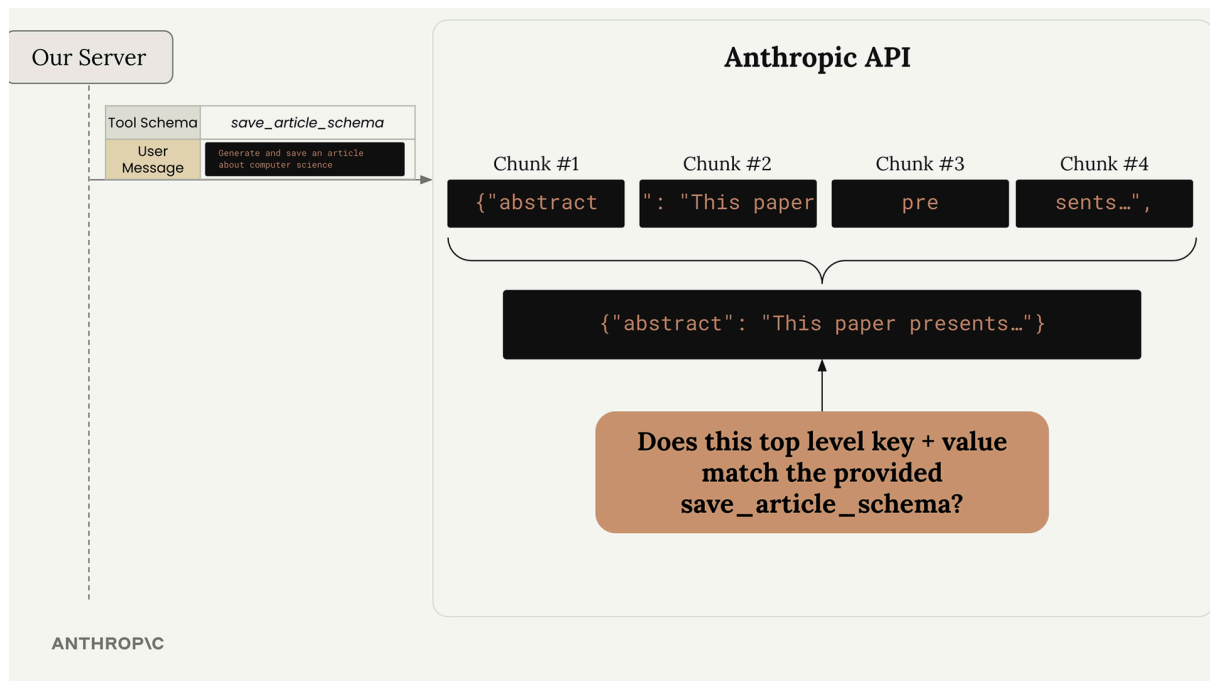


The API waits for complete top-level key-value pairs before sending anything. For example, if your tool expects this structure:

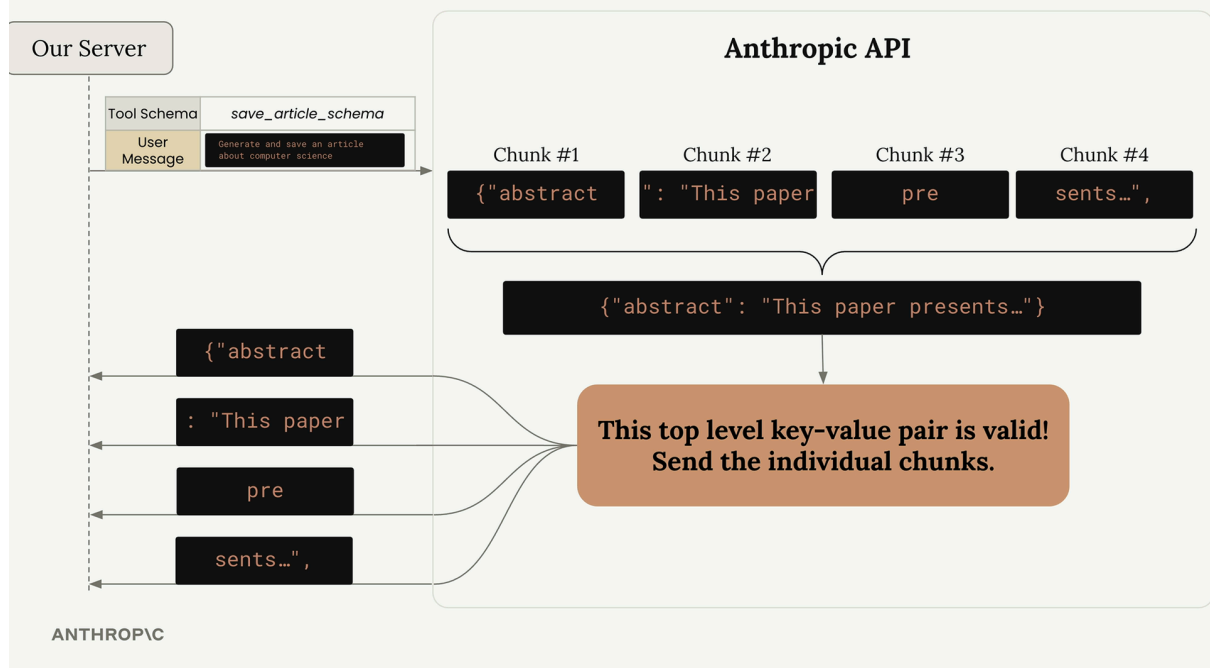
```
{  
  "abstract": "This paper presents a novel...",  
  "meta": {  
    "word_count": 847,  
    "review": "This paper introduces QuanNet..."  
  }  
}
```

The API will:

1. Wait until the entire **abstract** value is complete
2. Validate that key-value pair against your schema
3. Send all the buffered chunks for **abstract** at once
4. Repeat the process for the **meta** object



This validation process explains why you see delays followed by bursts of text, even with streaming enabled. The chunks are being held back until a complete, valid top-level key-value pair is ready.



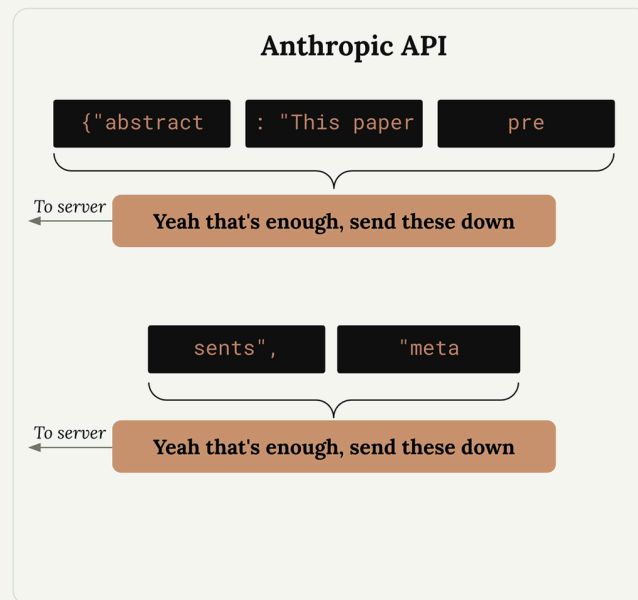
Fine-Grained Tool Calling

If you need faster, more granular streaming - perhaps to show users immediate updates or start processing partial results quickly - you can enable fine-grained tool calling.

Fine Grained Tool Calling

- Sometimes you want to get each chunk of generated JSON as fast as possible
- Fine grained tool calling sends **groups** of chunks without waiting for a full top level key to be created
- **Critical:** JSON validation is disabled! Your code should handle invalid tool inputs!

ANTHROPIC



Fine-grained tool calling does one main thing: it disables JSON validation on the API side. This means:

- You get chunks as soon as Claude generates them
- No buffering delays between top-level keys
- More traditional streaming behavior
- Critical: JSON validation is disabled - your code must handle invalid JSON

Enable it by adding **`fine_grained=True`** to your API call:

```
run_conversation(  
    messages,  
    tools=[save_article_schema],  
    fine_grained=True  
)
```

With fine-grained tool calling, you might receive a **word_count** value much earlier in the stream, without waiting for the entire **meta** object to be completed.

Handling Invalid JSON

When using fine-grained tool calling, Claude might generate invalid JSON like **"word_count": undefined** instead of a proper number. Your application needs to handle these cases gracefully:

```
try:
    parsed_args = json.loads(chunk.snapshot)
except json.JSONDecodeError:
    # Handle invalid JSON appropriately
    print("Received invalid JSON, continuing...")
```

Without fine-grained tool calling, the API's validation would catch this error and potentially wrap problematic values in strings, which might not match your expected schema.

When to Use Fine-Grained Tool Calling

Consider enabling fine-grained tool calling when:

- You need to show users real-time progress on tool argument generation

- You want to start processing partial tool results as quickly as possible
- The buffering delays negatively impact your user experience
- You're comfortable implementing robust JSON error handling

For most applications, the default behavior with validation is perfectly adequate. But when you need that extra responsiveness, fine-grained tool calling gives you the control to get chunks as fast as Claude can generate them.

The text edit tool

Downloads

005_text_editor_tool.ipynb

Important Note: Tool version strings can for all model versions can be found here:

<https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/text-editor-tool>

Claude comes with one built-in tool that you don't need to create from scratch: the text editor tool. This tool gives Claude the ability to work with files and directories just like you would in a standard text editor.

What the Text Editor Tool Can Do

The text editor tool provides Claude with a comprehensive set of file manipulation capabilities:

- View file or directory contents
- View specific ranges of lines in a file
- Replace text in a file
- Create new files
- Insert text at specific lines in a file
- Undo recent edits to files

The Text Editor Tool

- **Tool built directly into Claude**
- Allows Claude to create, read, and edit both directories and files
- Only the schema is build into Claude - you have to provide an implementation of the actual tool

ANTHROPIC

Gives Claude the ability to...

View file or directory contents

View specific range of lines in a file

Replace text in a file

Create new files

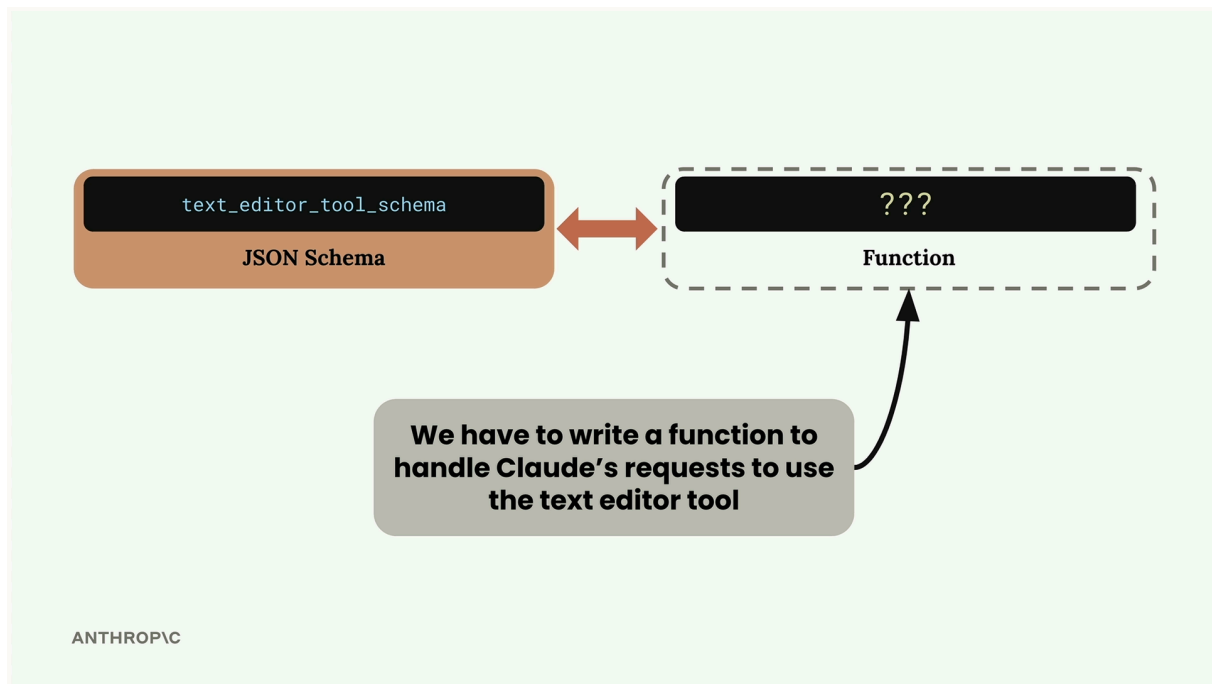
Insert text at specific lines in a file

Undo recent edits to files

This dramatically expands Claude's abilities and essentially gives it the power to act as a software engineer right out of the gate.

Understanding the Implementation Requirements

Here's where things get a bit confusing: while the tool schema is built into Claude, you still need to provide the actual implementation. Think of it this way - Claude knows how to ask for file operations, but you need to write the code that actually performs those operations.



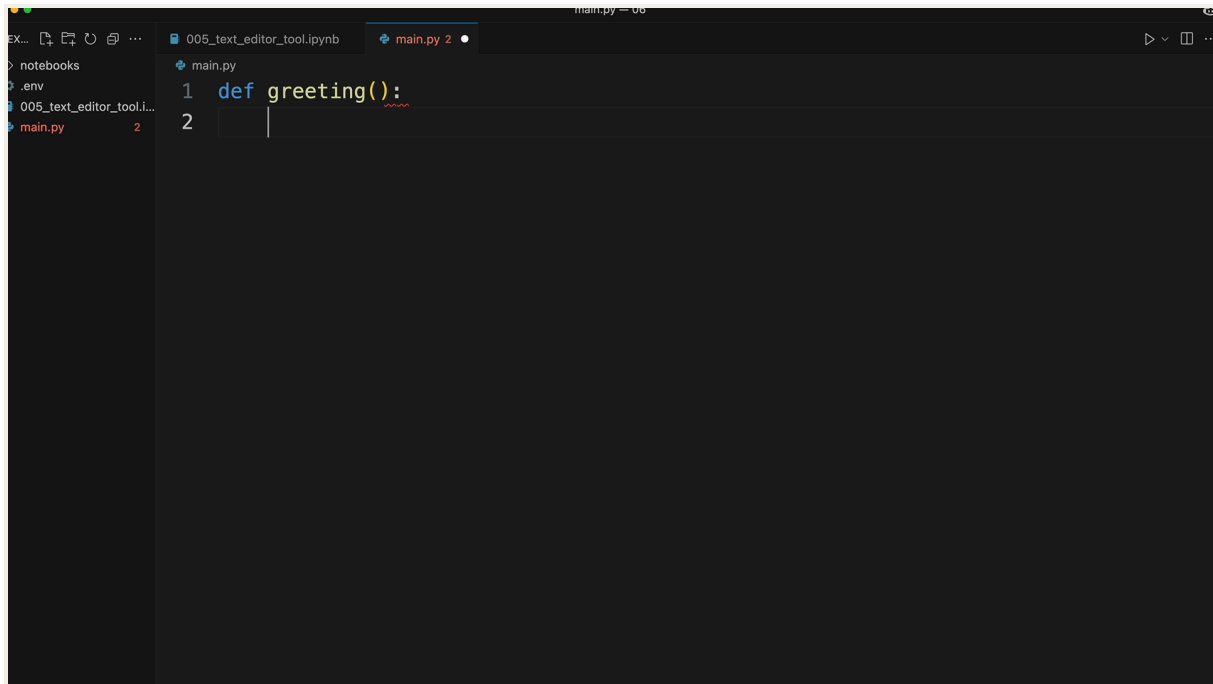
When you use other tools, you write both the JSON schema and the function implementation. With the text editor tool, Claude provides the schema knowledge, but you must write functions to handle Claude's requests to create files, read directories, replace text, and so on.

Schema Versions

While the main schema is built into Claude, you do need to include a small schema stub when making requests. The exact schema depends on which Claude model you're using:

```
def get_text_edit_schema(model):  
    if model.startswith("claude-3-7-sonnet"):  
        return {  
            "type": "text_editor_20250124",  
            "name": "str_replace_editor",  
        }  
    
```

```
elif model.startswith("claude-3-5-sonnet") :  
    return {  
        "type": "text_editor_20241022",  
        "name": "str_replace_editor",  
    }
```



Claude sees this small schema and automatically expands it into the full text editor tool specification behind the scenes.

Practical Example

Let's see the text editor tool in action. When you ask Claude to work with files, it will use the tool to read, modify, and create files as needed.

For example, if you ask Claude to "Open the ./main.py file and summarize its contents", Claude will:

1. Use the text editor tool to view the file
2. Read the contents
3. Provide you with a summary

You can take this further by asking Claude to modify files. For instance:
"Open the `./main.py` file and write out a function to calculate pi to the 5th digit. Then create a `./test.py` file to test your implementation."

Claude will:

1. View the existing `main.py` file
2. Replace its contents with a new implementation including the pi calculation function
3. Create a new `test.py` file with appropriate unit tests

Why Use the Text Editor Tool?

You might wonder why this tool exists when modern code editors already have AI assistants built in. The text editor tool becomes valuable in scenarios where:

- You're building applications that need to programmatically edit files
- You're working in environments without access to full-featured code editors
- You want to integrate file editing capabilities directly into your Claude-powered applications

Essentially, the text editor tool lets you replicate much of the functionality of a fancy AI-powered code editor within your own applications, giving you fine-grained control over how Claude interacts with your file system.

The web search tool

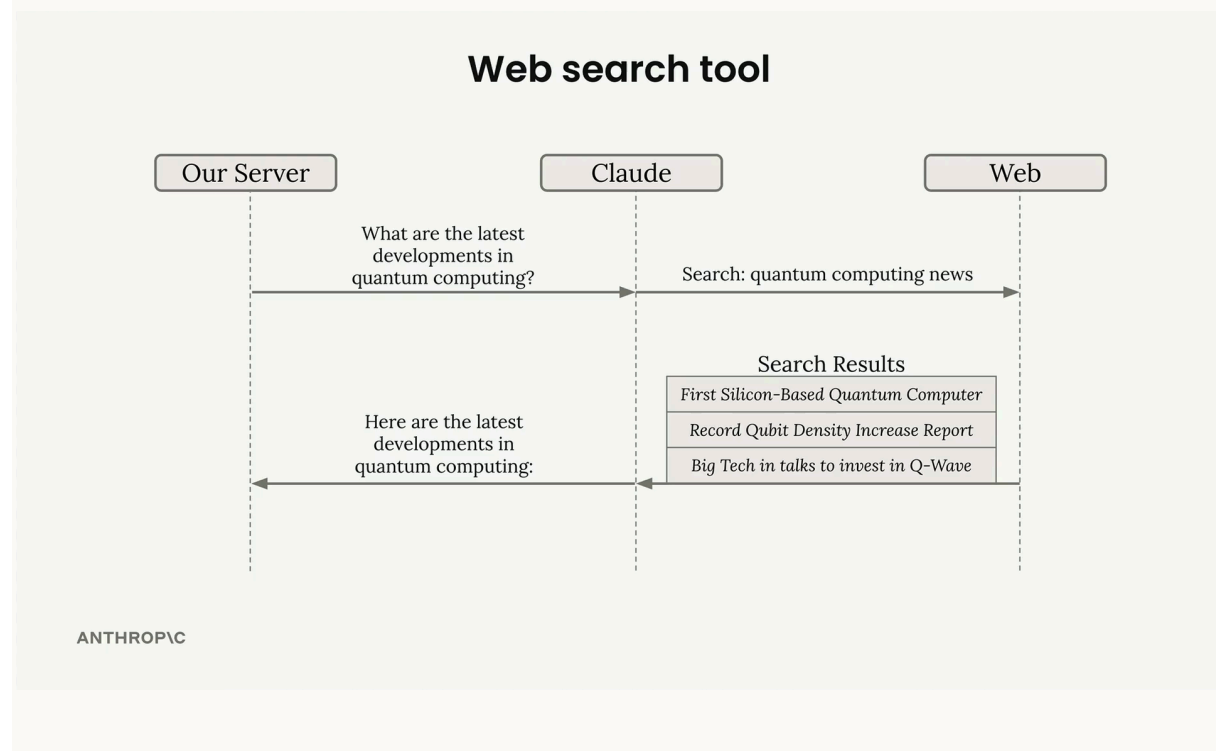
Downloads

006_web_search.ipynb (opens in new tab)

006_web_search_complete.ipynb

Important note: Your organization must enable the Web Search tool in the settings console before using it. You can find this setting here: <https://console.anthropic.com/settings/privacy>

Claude includes a built-in web search tool that lets it search the internet for current or specialized information to answer user questions. Unlike other tools where you need to provide the implementation, Claude handles the entire search process automatically - you just need to provide a simple schema to enable it.



Setting Up the Web Search Tool

To use the web search tool, you create a schema object with these required fields:

```
web_search_schema = {  
    "type": "web_search_20250305",  
    "name": "web_search",  
    "max_uses": 5  
}
```

The **max_uses** field limits how many searches Claude can perform. Claude might do follow-up searches based on initial results, so this prevents excessive API calls. A single search returns multiple results, but Claude may decide additional searches are needed.

How the Response Works

When Claude uses the web search tool, the response contains several types of blocks:

- Text blocks - Claude's explanation of what it's doing
- ServerToolUseBlock - Shows the exact search query Claude used
- WebSearchToolResultBlock - Contains the search results
- WebSearchResultBlock - Individual search results with titles and URLs
- Citation blocks - Text that supports Claude's statements

```

1  [
2      # Initial text block - Claude decides to use the web search tool
3      TextBlock(
4          text="I'll help you find information about the best exercises for building leg muscle.
5      ),
6
7      # Exact query Claude executed
8      ServerToolUseBlock(
9          input={
10             "query": "best exercises building leg muscle strength scientific research"
11         },
12     ),
13
14     # Search results that Claude found
15     WebSearchToolResultBlock(
16         content=[
17             WebSearchResultBlock(
18                 title="Gluteus Maximus Activation during Common Strength and Hypertrophy Exerc
19                 type="web_search_result",
20                 url="https://pmc.ncbi.nlm.nih.gov/articles/PMC7039033/",
21             )
22         ]
23     )
24 ]

```

The response structure lets you see exactly what Claude searched for and which sources it found. Citations include the specific text Claude used to support its answers, along with the source URLs.

Restricting Search Domains

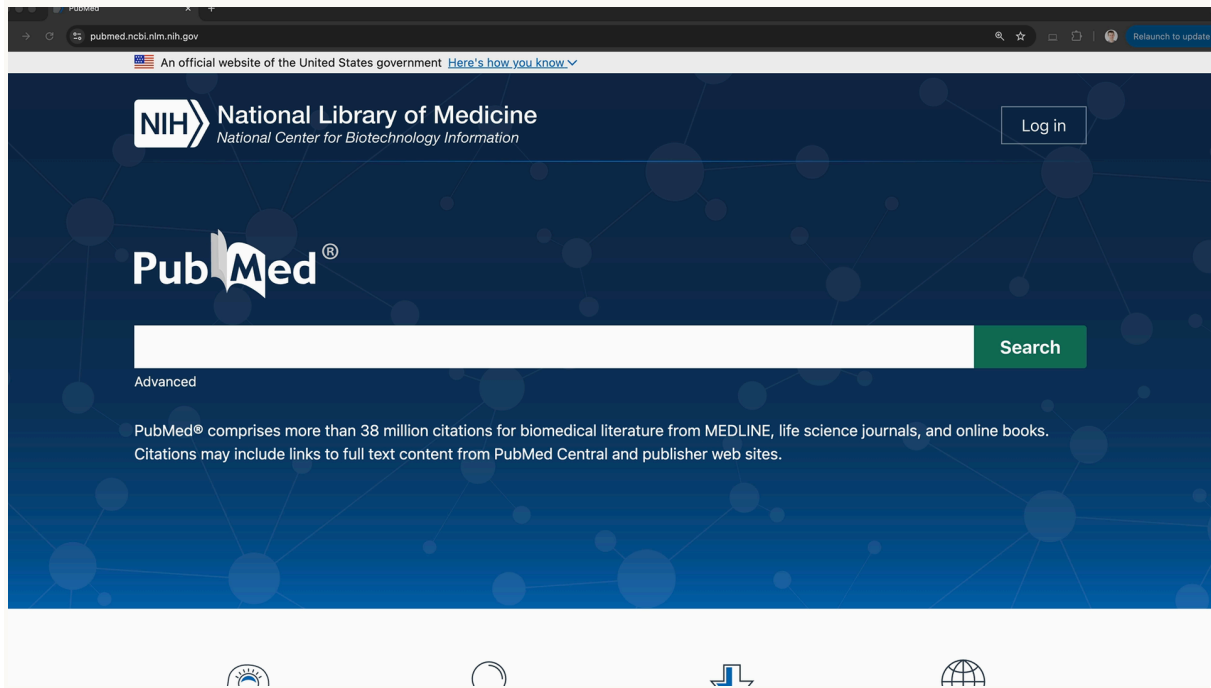
You can limit searches to specific domains using the **allowed_domains** field. This is particularly useful when you want reliable, authoritative sources:

```

web_search_schema = {
    "type": "web_search_20250305",
    "name": "web_search",
    "max_uses": 5,
    "allowed_domains": ["nih.gov"]
}

```

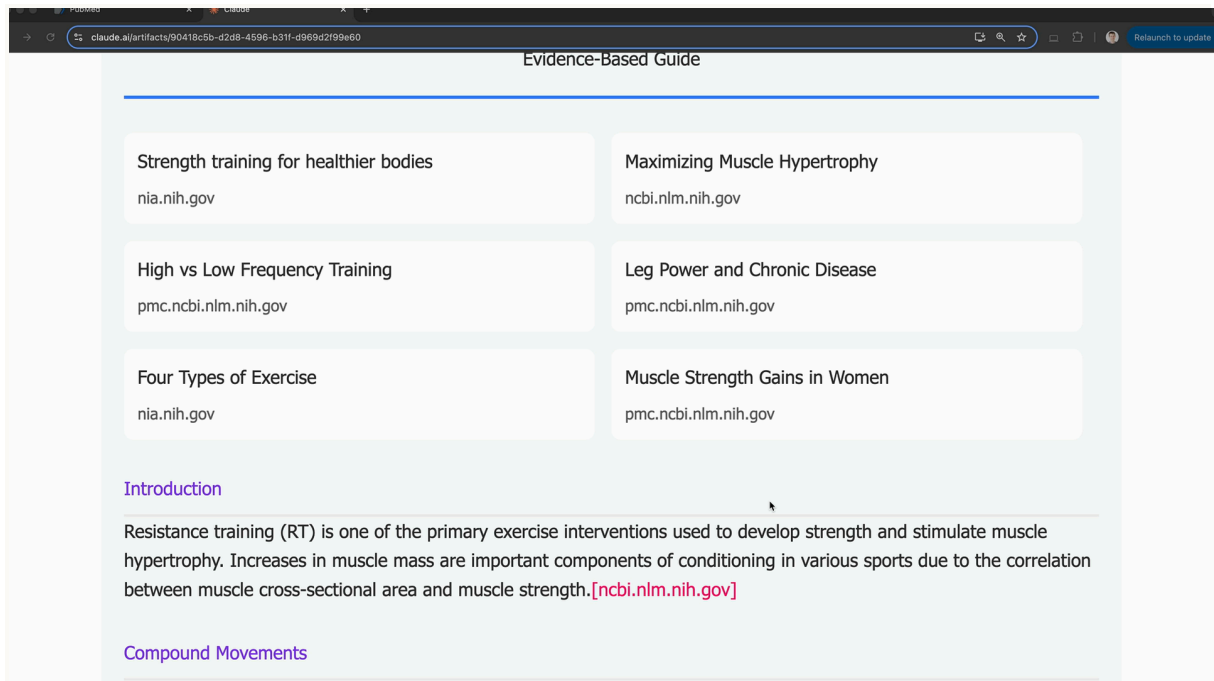
For example, when asking about medical or exercise advice, restricting to domains like PubMed (nih.gov) ensures you get evidence-based information rather than random blog content.



Rendering Search Results

The different block types in the response are designed for specific UI rendering:

- Render text blocks as regular content
- Display web search results as a list of sources at the top
- Show citations inline with the text, including the source domain, page title, URL, and quoted text



This structure helps users understand how Claude arrived at its answers and provides transparency about the sources being used. The citation format makes it clear which specific information came from which sources, building trust in the AI's responses.

Practical Usage

The web search tool works best for:

- Current events and recent developments
- Specialized information not in Claude's training data
- Fact-checking and finding authoritative sources
- Research tasks requiring up-to-date information

Simply include the schema in your tools array when making API calls, and Claude will automatically decide when a web search would help answer the user's question.